WOXSEN UNIVERSITY | SCHOOL OF TECHNOLOGY | DBMS COURSEWORK

Railway Ticket Reservation and Journey Management System

REVIEW 1: Problem Identification, Requirements, and Conceptual ER Design

Student Name: Heman | Roll Number: 25WU0102097 | Section: AIML Rhinos
Faculty Mentor: Prof. Sahithi | School of Technology, Woxsen University | Review 1

Problem Identification in Railway Journey Management

Core Technical Challenges in Multi-Hop Inventory

- **Monolithic vs Interval Seat Allocation:**
  Airline seats are occupied for the entire flight. In rail transport, a train visits 10+ stations. A seat can be booked from A-B, vacated, and immediately re-booked from B-D by another passenger.
- **Overlapping Segment Conflicts:**
  If Passenger 1 holds A-C, Passenger 2 cannot book B-D on the same seat because segment [B, C) is contested. Naive databases cause either catastrophic double-booking or severe capacity waste.
- **Coupling Definition vs Dated Runs:**
  Storing train definitions with travel dates causes severe redundancy and update anomalies.
- **Auditability & Financial Integrity:**
  Cancellations, refunds, and queue promotions require transactional consistency across multiple tables.

Relational Database Solution Approach

- **Segment-Wise Relational Modeling:**
  Model seats over half-open intervals [origin_seq, dest_seq) in a dedicated journey_segments table.
- **Mathematical Overlap Formula:**
  Enforce s < r_end AND r_start < e across active allocations.
- **ACID Transaction Isolation:**
  Use row-level locking (SELECT ... FOR UPDATE) to serialize competing requests on candidate seats.
- **Third Normal Form (3NF) Schema:**
  Separate stations, routes, trains, dated runs, coaches, seats, passengers, bookings, and payments into 14 distinct relations.

Project Objectives & Scope Definition

1. Relational Database Design

✔ Develop an enterprise schema with 14 normalized relations in 3NF.

✔ Implement primary keys, foreign keys with ON DELETE rules, and domain CHECK constraints.

✔ Ensure entity decoupling: stations, ordered stops, locomotive definitions, and calendar runs.

✔ Use exact monetary DECIMAL(10,2) types to eliminate floating-point rounding errors.

2. Business Logic & Integrity

✔ Implement half-open interval seat allocation [origin_seq, dest_seq).

✔ Guarantee transaction safety via pessimistic row locking (FOR UPDATE).

✔ Deterministic fare calculation using distance rates and travel class multipliers.

✔ Automated FIFO waitlist queue allocation and promotion upon cancellation.

3. Project Scope Boundaries

INCLUDED IN PROTOTYPE:

- Full relational schema in MySQL 8.x / SQLite
- Interactive web UI with train search & booking
- 12 Advanced SQL reports and analytical queries
- Automated pytest suite covering all integrity rules

EXCLUDED FROM SCOPE:

- Live external Indian Railway API integration
- Real banking payment gateway (UPI is simulated)
- Hardware ticketing kiosk devices

Stakeholders and System User Roles

Passenger / Traveler

Role Type: End-Users

✔ Search trains between any two stations on selected dates.

✔ Check live seat availability by travel class (CC, 1A, 2A, 3A, SL).

✔ Book tickets with berth preferences and receive 10-digit PNR.

✔ Check PNR status, view ERS slip, and check waitlist position.

✔ Cancel confirmed or waitlisted bookings and claim refund.

Reservation Operator

Role Type: Ticket Counter Staff

✔ Issue counter tickets on behalf of walk-in passengers.

✔ Lookup passenger profiles and manage ticket cancellations.

✔ Monitor live coach occupancy rates prior to chart preparation.

✔ Verify passenger identification credentials (Aadhaar, Passport).

Database Administrator

Role Type: System & Schema Admins

✔ Manage station nodes, train catalog definitions, and route stops.

✔ Configure operational train runs and assign coach compositions.

✔ Maintain tariff rules (base fare, rate/km, reservation fees).

✔ Execute database audits, integrity checks, and SQL reports.

System Requirements Specification

Functional Requirements (FR)

- FR-01 Train Discovery: Search trains between origin and destination on a travel date; enforce origin stop sequence < destination stop sequence.
- FR-02 Segment Availability: Compute real-time free seats per travel class using the interval non-overlap condition.
- FR-03 Ticket Reservation: Atomic booking generation with unique 10-digit PNR, berth assignment, and payment record.
- FR-04 Waitlist Management: Automatic placement into a sequenced queue when segment capacity is exhausted.
- FR-05 Cancellation & Refund: Process voluntary cancellations, apply policy deductions (20%), and execute waitlist promotion.
- FR-06 SQL Reporting: Execute 12 analytical queries including occupancy, revenue reconciliation, and route demand.

Non-Functional Requirements (NFR)

- NFR-01 Data Integrity (ACID): All updates wrapped in transactions; rollback on any failure; declarative foreign keys and CHECK constraints.
- NFR-02 Concurrency Safety: Serialize concurrent bookings using InnoDB row-level locking (SELECT ... FOR UPDATE) to eliminate race conditions.
- NFR-03 Financial Precision: Store monetary amounts strictly as DECIMAL(10,2) to eliminate floating-point rounding errors.
- NFR-04 Performance & Indexing: B-Tree indexes on PNR, station codes, train numbers, and segment allocation intervals for sub-second responses.
- NFR-05 Usability & Responsiveness: Intuitive browser interface formatted like an official IRCTC electronic reservation slip.

Core Business Rules & Invariants

Formal Business Invariants Enforced by Database & Application

BR-1: Train Catalog vs Run Decoupling: A train represents master route metadata; a train run represents an operational departure on a specific date. Bookings on different dates for the same train number must never conflict.

BR-2: Route Progression Invariant: Stations must be sequenced with positive integers (1, 2, 3...). A valid journey requires origin_stop_seq < destination_stop_seq.

BR-3: Segment Interval Exclusion: A confirmed reservation allocates a seat over half-open interval [s, e). Two allocations on the same seat overlap iff existing.start < req.end AND req.start < existing.end. Overlapping requests must be rejected.

BR-4: Non-Overlapping Seat Reuse: Two distinct passengers may hold the exact same physical seat on the same train run provided their journey intervals are disjoint (e.g., Passenger A on [1, 2) and Passenger B on [2, 4)).

BR-5: Waitlist Serialization: Waitlist entries are scoped by (run_id, travel_class) and assigned sequential queue positions (1, 2, 3...) using atomic transactions.

BR-6: Deterministic Cancellation Policy: Voluntary cancellation of a confirmed ticket retains 20% cancellation fee and disburses 80% refund. Waitlisted tickets deduct a flat ₹30 clerkage fee.

BR-7: Atomic Waitlist Promotion: Cancelling a confirmed seat immediately checks the waitlist queue in FIFO order, promoting the first candidate whose requested interval is satisfied, updating both records in one transaction.

Conceptual Entity-Relationship (ER) Diagram

ER Model Architecture: Entities, Cardinalities & Junctions

STATION
PK: station_id
station_code (UQ)
station_name, city

ROUTE
PK: route_id
route_code (UQ)
src_stn, dst_stn

TRAIN
PK: train_id
train_number (UQ)
train_name, route_id

TRAIN_RUN
PK: run_id
train_id (FK)
run_date (UQ), status

TRAIN_STOP
PK: stop_id
route_id, station_id
stop_sequence (UQ)

COACH
PK: coach_id
train_id, coach_code
travel_class, seats

SEAT
PK: seat_id
coach_id (FK)
seat_number, berth

PASSENGER
PK: passenger_id
full_name, age, gender
email, id_card

BOOKING
PK: booking_id
pnr_number (UQ)
run_id, passenger_id
orig_stn, dest_stn, fare

JOURNEY_SEGMENT
PK: segment_id
booking_id, seat_id, run_id
[start_seq, end_seq)

WAITLIST
PK: waitlist_id
booking_id, run_id
class, position, status

Entity Responsibilities and Cardinalities

Entity Inventory and Cardinality Matrix

Entity Name	Type	Primary Key & Unique Attributes	Cardinality	Core Responsibility
Station	Master	PK: station_id | UQ: station_code	1 : N with Train_Stop	Physical geographic railway passenger nodes
Route	Master	PK: route_id | UQ: route_code	1 : N with Train, Train_Stop	Track corridors linking source and destination
Train	Master	PK: train_id | UQ: train_number	1 : N with Train_Run, Coach	Static train catalogs (Vande Bharat, Rajdhani)
Train_Run	Operational	PK: run_id | UQ: (train_id, run_date)	1 : N with Booking, Segment	Dated calendar train departures (3NF Decoupling)
Coach & Seat	Inventory	PK: coach_id, seat_id | UQ: seat_num	1 : N Coach to Seat	Rolling stock physical seating units & berth types
Passenger	Profile	PK: passenger_id | UQ: email	1 : N with Booking	Registered traveler identity and contact records
Journey_Segment	Allocation	PK: segment_id	M : 1 with Booking, Seat, Run	Interval seat allocation [start_seq, end_seq)

Initial Relational Schema Notation

Formally Specified Relational Schema (14 Relations)

1. STATIONS (station_id PK, station_code UNIQUE, station_name, city, state)
2. ROUTES (route_id PK, route_code UNIQUE, source_station_id FK, destination_station_id FK, total_distance_km)
3. TRAIN_STOPS (stop_id PK, route_id FK, station_id FK, stop_sequence, distance_from_source_km, arrival_time, departure_time)
4. TRAINS (train_id PK, train_number UNIQUE, train_name, train_type, route_id FK)
5. TRAIN_RUNS (run_id PK, train_id FK, run_date, status) [UNIQUE: train_id, run_date]
6. COACHES (coach_id PK, train_id FK, coach_code, travel_class, total_seats) [UNIQUE: train_id, coach_code]
7. SEATS (seat_id PK, coach_id FK, seat_number, berth_type) [UNIQUE: coach_id, seat_number]
8. PASSENGERS (passenger_id PK, full_name, gender, age, email, phone, id_card_type, id_card_number)
9. FARE_RULES (fare_id PK, train_type, travel_class, base_fare, rate_per_km, reservation_fee) [UNIQUE: train_type, travel_class]
10. **BOOKINGS (booking_id PK, pnr_number UNIQUE, run_id FK, passenger_id FK, origin_station_id FK, destination_station_id FK, origin_stop_seq, destination_stop_seq, travel_class, booking_status, total_fare, booked_at)**
11. **JOURNEY_SEGMENTS (segment_id PK, booking_id FK, seat_id FK, run_id FK, start_stop_seq, end_stop_seq, allocation_status)**
12. WAITLISTS (waitlist_id PK, booking_id FK, run_id FK, travel_class, origin_stop_seq, destination_stop_seq, position, waitlist_status, created_at)
13. PAYMENTS (payment_id PK, booking_id FK, amount, payment_method, transaction_reference UNIQUE, payment_status, paid_at)
14. CANCELLATIONS (cancellation_id PK, booking_id FK UNIQUE, cancellation_reason, cancellation_fee, refund_amount, refund_status, cancelled_at)

Proposed Full-Stack System Architecture

Multi-Tier Relational Application Data Flow

Presentation Layer

HTML5, CSS3, Bootstrap 5
Jinja2 Server Templates
Responsive Ticket UI

Application Layer

Python 3 & Flask Framework
Blueprint Route Handlers
Server-side Validation

Business Service Layer

Availability Service [s, e)
Transaction Booking Engine
Waitlist Promotion & Refunds

Persistence Layer

MySQL 8.x InnoDB RDBMS
SQLAlchemy ORM + Raw SQL
ACID Locks & 12 Views

Implementation & Review Roadmap

Three-Stage Milestone Plan (Week 7 to Week 16)

🎯 Review 1 (Week 7) - Conceptual Design & Requirements [COMPLETED]

Finalize business rules, stakeholder requirements, ER modeling, entity inventory, and initial relational schema mapping.

🎯 Review 2 (Week 11) - Normalization, DDL & SQL Implementation [NEXT STAGE]

Derive functional dependencies, prove 3NF normalization, deploy MySQL 8.x schema DDL with InnoDB constraints, seed benchmark records, and execute 12 analytical SQL queries.

🎯 Review 3 (Week 16) - Final Implementation, Testing & Live Demo [FINAL STAGE]

Connect Flask web frontend with live database, verify segment-wise allocation and automated waitlist promotion, execute 16 automated pytest suites, and deliver final project reports.